FPS-3000 — VersaBus card trace worksheet

For the V-BUS XLTR (612-4803-400-G) and AP I/F (612-4448-401-F). Rebuilt 2026-07-29; supersedes the earlier edition.

Every expectation is a PREDICTION from the SBC firmware, not an observation. Where a check disagrees, the board wins - record what you find. Sources: refs_extracted/versabus_access_map.md, MC68153 datasheet, versabus_pinout.md.

Check 0 — the phase beacon: read the machine's own self-test progress

Costs nothing and needs no debugger. Every power-on self-test writes phase<<8 | subtest to CHANNEL_SELECT ($FF0204) before it runs. Scope, latch or read that register during reset: the LAST value before a hang names the failing subtest. The suite runs $0100 through $2903 on a healthy machine and ends in the RTOS idle loop.

beacon	what it was testing when it stopped
$01xx (105)	CPU flags / arithmetic - fails before any chassis access
$02xx (6)	$1FFF1 bit 6 <-> $F70019 bit 3 inverse wiring
$0300 (1)	ROM decode
$0400 (1)	RAM address decode / byte lanes
$0500 (1)	board status not reaching (x & $3F31) == $3F11
$06xx (8)	VMOD control register pattern round-trip
$0700 (1)	short-I/O window at $F82000 must BUS ERROR
$08xx (4)	I/O channel
$09xx (5)	MC6840 PTM, its IRQ path, or $1FFF1 bit 7 gating
$1000 (1)	address-space boundary map
$11xx-$14xx	panel-bus interrupts and vectors
$15xx (6)	CHANNEL_SELECT must be a clean read/write register
$1600 (1)	XLTR register file, or the BIM population bit
$17xx (4)	DATA_HI page gating on the $400000 window
$18xx (4)	XLTR mode/page register $FF0216
$19xx (5)	XLTR data register $FF0214
$1Axx (4)	XLTR status/IRQ $FF0218 + panel command port
$20xx-$23xx	RAM test, FIRST pass: $000400-$01F000
$24xx-$27xx	RAM test, SECOND pass: $010000-$01F000 = the WCS staging buffer
$28xx (26)	CHASSIS MEMORY test - 65k window accesses despite few beacon subtests
$29xx (32768)	CHASSIS MEMORY test via the $400000 paged window (131k accesses), PTM alongside

(n) = number of subtests in that phase, measured. 30 phases exist: $01-$09, $10-$1A, $20-$29 (BCD, plus $1A).

PHASE $29 IS 99.4% OF THE RUN (32,768 of 32,967 beacon writes). The beacon SITTING in $29xx is NORMAL, not a hang.

Reaching $24xx but hanging in $25xx-$27xx = good low RAM, bad WCS staging RAM. That split is the useful one.

Beacon observed: _____ Boot completes to idle loop? Y / N

Check 1 — identify the three BIM packages

Look for three 40-pin DIPs. MC68153 is expected; any part with the same pinout is equally consistent.

The parts survey did NOT find them - this is a genuine open question, not a formality.

package ref	part number as marked	date code	pin 1 OK?

Confirm before trusting: pin 3 = CS, pins 38/39/40 = A1/A2/A3, pins 28,29,32-37 = D0-D7, pin 18 = CLK, pin 4 = DTACK.

Check 2-4 — interrupt level, daisy chain, address decode

Check 2 — the interrupt level

The firmware writes $5F to five channel control registers and $5E to one. Per the MC68153 datasheet bits 0-2 choose which IRQ output the chip asserts, so $5F = IRQ7 and $5E = IRQ6. Buzz the pins and the question closes.

BIM IRQ pin	signal	VersaBus P1 pin	connected? Y/N	notes
8	IRQ1*	87		
12	IRQ2*	88		
13	IRQ3*	89		
14	IRQ4*	90		
15	IRQ5*	91		
16	IRQ6*	92		
17	IRQ7*	93		

Prediction: pin 17 wired on the BIMs at $FF0240 and $FF0250; pin 16 on the one at $FF0230. If IRQ5 is the connected pin instead, the datasheet reading is wrong. Either result is useful - record which pins go anywhere at all.*

Check 3 — the IACK daisy chain

Five enabled channels request the same level, so IACKIN*/IACKOUT* decides who wins a tie. Establish the order.

from	pin	to	pin	connected?
BIM ___ IACKOUT*	7	BIM ___ IACKIN*	6	
BIM ___ IACKOUT*	7	BIM ___ IACKIN*	6	
VersaBus IACKIN		first BIM IACKIN*	6	
last BIM IACKOUT*	7	VersaBus / next card		

Record the physical left-to-right order on the card as well as the electrical order.

Check 4 — address decode and chip select

Each BIM answers a 16-byte window; A1-A3 select the register inside, so A4 and above decode into CS.

window	registers	A6	A5	A4	which package has this CS?
$FF0230-$FF023F	BIM0 CR0-3, VR0-3	0	1	1	
$FF0240-$FF024F	BIM1 CR0-3, VR0-3	1	0	0	
$FF0250-$FF025F	BIM2 CR0-3, VR0-3	1	0	1	
$FF0200-$FF021B	XLTR control file	0	0	0	

A4 and A5 alone do NOT separate the windows: BIM1 and the XLTR file both read A4=0, A5=0. A6 is the deciding bit.

Check 5-6 — register layout, interrupt sources

Check 5 — register layout confirmation

Inferred from firmware writes: CRn and VRn are the same channel. If the board disagrees, the whole channel-to-task mapping in the access map is wrong.

offset	A3	A2	A1	register	channel	firmware writes
+$0	0	0	0	CR0	ch0	$5E or $5F or $00
+$2	0	0	1	CR1	ch1	$00
+$4	0	1	0	CR2	ch2	$5F
+$6	0	1	1	CR3	ch3	$5F or $00
+$8	1	0	0	VR0	ch0	vector number
+$A	1	0	1	VR1	ch1	vector number
+$C	1	1	0	VR2	ch2	vector number
+$E	1	1	1	VR3	ch3	vector number

The address arrives as A01-A03 and reaches the BIM as A1-A3 (pins 38,39,40). Confirm no swap: reversed A1/A3 would exchange the CR and VR halves and everything above would still look self-consistent.

Check 6 — who drives the device interrupt inputs

The four INT inputs are what the chassis pulls to request service. Tracing them backwards names each task's owner.

BIM	INT pin	name	firmware owner (predicted)	what actually drives it
BIM1	23	INT2	TCBXP1I (XP-32 channel 1)	
BIM1	24	INT3	TCBXP2I	
BIM2	19	INT0	TCBXP3I	
BIM2	22	INT1	TCBXP4I	
BIM2	23	INT2	TCBIO1I (host link)	
BIM0	19	INT0	TCBRDHC, vector $41	

The last row matters most. Vector $41 reaches F04930, the handler that receives the chassis command stream - whatever drives BIM0 INT0 is the chassis's request line for SBC service.

Check 7-8 — AP I/F windows, the Am29116 pair

Check 7 — AP I/F channel windows [CORRECTED - the earlier edition had three of four roles wrong]

Four windows on a 32-byte stride. The card carries eight Am29705 16x4 TWO-PORT RAMs = 32 bits of width.

offset	ch1	ch2	ch3	ch4	role
+$04	$FF0044	$FF0064	$FF0084	$FF00A4	write port
+$08	$FF0048	$FF0068	$FF0088	$FF00A8	32-bit data, HIGH half
+$0A	$FF004A	$FF006A	$FF008A	$FF00AA	32-bit data, LOW half
+$0E	$FF004E	$FF006E	$FF008E	$FF00AE	command/trigger, $8000 fires

+$08 and +$0A are ONE 32-bit register, not data + status. $FF0048 is never read by the ROM. Do NOT look for a '$4F status value' - that came from our own emulator, not the hardware, and the earlier edition was wrong to ask.

question	finding
Am29705 designators (expect 8)	
Do all four channels share them, or silicon per channel?	
Ribbon connector 1: pin count / signals	
Ribbon connector 2: pin count / signals	
Which pins carry the host-side 32-bit path?	

The host-side counterpart card is missing, so its pinout has to come off this card.

Check 0c - stack high-water mark, from one memory dump

The supervisor stack top is $0800 and the kilobyte below it is pre-filled with $09ABCDEF, from $0404 up.
Dump that range and count surviving $09ABCDEF longwords. 236 of 256 = normal (a clean boot uses 76 bytes).
A low count means something recursed. ZERO means the stack has already overrun into the vector table at $03FF,
which sits directly below with no guard region - and that ends in a double bus fault.

Surviving $09ABCDEF longwords: _____ of 256 Deepest address reached: $_____

Check 0b - if the board dies, the panel port names the exception

The last word written to $FF000E before a hang is a CPU exception code. Nine of them, from a table at $F0A23A.
This costs nothing to watch and immediately separates a bus fault from a stray interrupt.

code	exception	code	exception
$29E	bus error (2)	$2A3	TRAPV (7)
$29F	address error (3)	$2A4	privilege violation (8)
$2A0	illegal instruction (4)	$2A5	uninitialised interrupt (15)
$2A1	divide by zero (5)	$2A6	CATCH-ALL - stray IRQ on any unused vector
$2A2	CHK (6)	$2B2	kernel-region fatal stub at $F001A0

$2A6 is the one to expect from a wiring fault: it is installed on 182 unused user vectors.

Last $FF000E value observed: _____

Check 7c — two predictions this firmware makes that a bus trace can falsify

Both follow from decoding alone and neither has been checked against hardware. They are cheap to test and each would settle an open question outright.

1. $10AA MUST BE WRITTEN BY SOMETHING OTHER THAN THE CPU. TCBIO1I dispatches on it, but the only code that writes that array is the host-command-1 handler, which indexes $10A0 by (channel-1)*2 and validates 1 <= channel <= $105E. Reaching $10AA needs channel 6; $105E counts nonzero ports among exactly FOUR. So the CPU cannot write it. PREDICTION: a bus trace shows a NON-CPU master writing $0010AA. If instead nothing ever writes it, the host path is dead code in this ROM revision.
2. A PANEL COMMAND ISSUED FROM A CHANNEL ISR CANNOT COMPLETE. The issuer ends in bra . and is only released by the responder on BIM0 ch0, which the firmware programs to LEVEL 6 (CR=$5E), while every channel ISR runs at LEVEL 7 (CR=$5F) and never lowers SR. PREDICTION: if the chassis answers $281 via BIM0 ch0, the SBC hangs at $F05E86. If the machine instead works, the response arrives by some other path - find it. SEVERITY: level 7 is NON-MASKABLE on a 68000, so if the channel interrupt re-asserts while the SBC is spinning, the ISR re-enters and each entry pushes a frame onto a 400-byte task stack. In emulation that overruns the ENTIRE vector table ($104-$3FF, 729 bytes) and would end in a double bus fault + HALT on iron. So the symptom to look for is not a quiet hang but FAIL+HALTED - which is what the first burn showed.

Trace result 1: _____ Trace result 2: _____

Check 7b — bus mastership

The SBC never asserts a VersaBus transfer request: $1FFF0, the control register half holding Transfer Request and Block Transfer Request, is written $00 every time. So the CHASSIS masters the bus and DMAs into SBC RAM.

question	finding
Which card drives BR*/BGACK* on P1?	
Does the AP I/F or the XLTR contain the DMA address counter?	
What writes SBC RAM $10AA and $105E? (firmware never does)	

Check 8 — the two Am29116 on the XP32 EXEC card

The owner counts two; the photo survey read one. Settle it before any PROM comes off - it decides whether a dump holds one instruction stream or two.

question	why it matters	finding
How many Am29116 packages?	survey says 1, owner says 2	
One shared 16-bit field, or 16 each of them?	one or two 80?	
Carry / status link between them?	cascaded = one 32-bit ALU	